Phase 1: Mid-Semester Project (The MVP) - Code Use

STUDENT VIEW

8ce00921-f923-41fe-8657-6b34a3f5913b

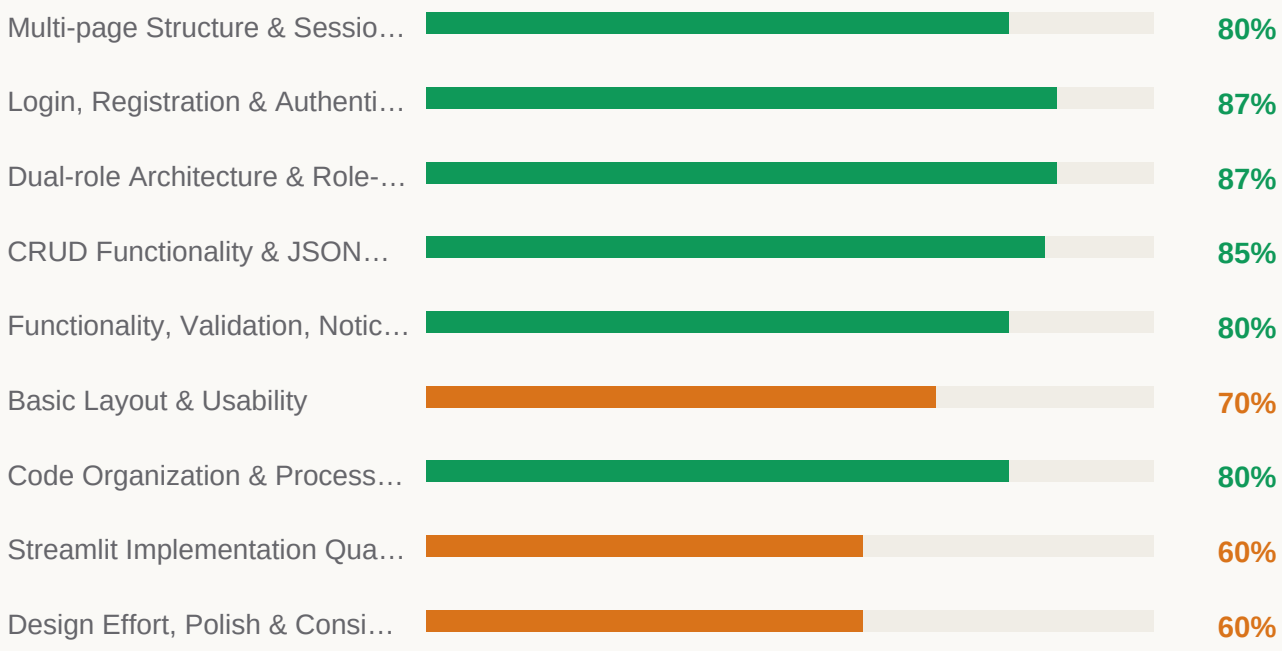
Attempt 4 • Apr 4, 2026, 5:50 PM • Apr 5, 2026, 11:03 PM

79/100

Solid mid-MVP Streamlit app with functioning multi-page navigation, persistent user and expense JSON storage, and role-based admin features. Core CRUD and auth flows work, though some UI polish, robust validation, and Streamlit best-practice improvements are needed. The app is usable and demonstrates clear separation of primary features but has small reliability/usability gaps to address.

Rubric Overview

Score distribution across all criteria



Rubric Breakdown

Multi-page Structure & Session-State Navigation

8 / 10

You implemented reliable multi-page navigation using st.session_state and st.rerun(), and the sidebar navigation flows work for logged-in users. Consider adding clearer initializations and guards so pages can't be reached with missing session data (e.g., protect edit page when edit_expense is absent).

Login, Registration & Authentication Flow

13 / 15

Registration and login use persistent JSON and include sensible validation (email format, password length, duplicate check). Improve by hashing passwords, handling malformed users.json more robustly, and showing clearer post-registration routing and confirmations.

Dual-role Architecture & Role-based Feature Structure

13 / 15

App supports admin and user roles with correct routing and admin-only pages for management actions. To strengthen, expand role-differentiated features for non-admins and centralize role checks to avoid accidental access.

CRUD Functionality & JSON Persistence

17 / 20

Create, read, update, and delete operations persist to JSON and reflect in the UI after rerun; admin delete and edit work end-to-end. Ensure JSON read/write is consistently wrapped (avoid potential missing variables if load fails) and confirm types (amount) are stored/loaded reliably.

Functionality, Validation, Notices & User Flow

8 / 10

Main workflows complete and show success/error notices; login, add, edit, and delete show feedback. Add more validation (e.g., check edit inputs, handle missing edit state) and friendly failure messages for JSON errors or unexpected states.

Basic Layout & Usability

7 / 10

Layout uses sidebar and columns which improves readability and dashboard-like presentation. Improve spacing, container consistency, and avoid showing raw data dumps; make forms and dashboards visually consistent and add headings/subsections where helpful.

Code Organization & Process Separation

4 / 5

Code is fairly readable and split by page sections; helper functions exist for validation and expense lookup. Extract repeated JSON I/O, page renderers, and CRUD operations into functions/modules to reduce inline logic and improve maintainability.

Streamlit Implementation Quality & Best Practices

3 / 5

You use st.session_state and st.rerun appropriately and assign widget keys in several places, which stabilizes behavior. Strengthen by adding explicit keys everywhere, pre-filling edit widgets, avoiding possible duplicate keys in dynamic loops, and handling missing session_state before creating widgets.

Design Effort, Polish & Consistency (Design Excellence)

6 / 10

The app demonstrates thoughtful layout choices (sidebar, metrics, charts) and a dashboard feel. To reach a higher polish, refine spacing, consistent container borders/typography, and unify visual hierarchy across pages and roles.

Strengths

- Working multi-page navigation using st.session_state with clear pages and rerun control.
- Persistent JSON storage for users and expenses with read/write on create/update/delete actions.
- Role-based routing and admin-specific features (view, edit, delete) implemented and enforced.

Needs Attention

- Improve defensive checks (e.g., ensure expenses/users lists are loaded before use and handle malformed JSON more explicitly).
- Make form state initialization consistent when editing (prefill edit inputs from session_state and use keys to avoid stale widgets).
- Enhance input validation and error handling (e.g., sanitize numeric types from JSON, validate edit inputs, and handle missing edit_expense).
- Polish layout and consistent use of containers/columns and avoid dumping backend data structures directly to UI in raw form.